Jour 2 : Tokenization et Vectorisation

Transformer le texte en representations numeriques exploitables

Tokenization : definition et methodes

Tokenization est le processus de decouper le texte en unites distinctes (tokens). C'est une etape critique qui determine la granularite de l'analyse.

Sentence Tokenization

Decouper le texte en phrases.

Input: "Bonjour. Comment allez-vous?"

Output: ["Bonjour.", "Comment allez-vous?"]

Word Tokenization

Decouper en mots individuels.

Input: "C'est magnifique!"

Output: ["C'est", "magnifique", "!"]

Subword Tokenization

Decouper en sous-mots/caracteres.

Input: "unbelievable"

Output: ["un", "##believ", "##able"]

PYTHON

```python
# Word tokenization avec NLTK
from nltk.tokenize import word_tokenize, sent_tokenize

texte = "Bonjour! C'est un exemple. Comment allez-vous?"

# Sentence tokenization
phrases = sent_tokenize(texte, language='french')
print(phrases)
# Output: ["Bonjour!", "C'est un exemple.", "Comment allez-vous?"]

# Word tokenization
mots = word_tokenize(texte, language='french')
print(mots)
# Output: ["Bonjour", "!", "C", "'", "est", "un", "exemple", ".", "Comment",
"allez", "-", "vous", "?"]
```

Bag of Words (BoW)

Bag of Words est l'approche la plus simple pour vectoriser du texte : transformer chaque document en vecteur de decompte des mots.

Exemple simple

Document	python	machine	learning	donnees
Doc 1: "python machine learning"	1	1	1	0
Doc 2: "python et donnees"	1	0	0	1
Doc 3: "machine learning et donnees"	0	1	1	1

PYTHON

```python
from sklearn.feature_extraction.text import CountVectorizer

# Documents
docs = [
    "python machine learning",
    "python et donnees",
    "machine learning et donnees"
]

# Create CountVectorizer
vectorizer = CountVectorizer(lowercase=True)
X = vectorizer.fit_transform(docs)

print("Vocabulaire:", vectorizer.get_feature_names_out())
print("Matrice sparse shape:", X.shape)
print("Document 1 (dense):", X[0].toarray())
```

> ⚠ **Limitation majeure :** Bag of Words ignore l'ordre des mots ("un bon film" vs "un film bon mauvais") et la semantique profonde. Tous les mots ont le meme poids.

⚖️ TF-IDF : Introduction

TF-IDF (Term Frequency - Inverse Document Frequency) ameliore Bag of Words en donnant plus de poids aux mots rares et informatifs.

🧮 Term Frequency (TF)

Frequence du mot dans le document.

TF(t, d) = nombre d'occurrences de t dans d

Exemple: Si "python" apparat 5 fois dans un document de 100 mots, TF = 5/100 = 0.05

🌐 Inverse Document Frequency (IDF)

Rareté du mot dans le corpus entier.

IDF(t) = log(N / df(t))

Ou N = nb total de docs, df(t) = nb de docs contenant t

Score final : TF-IDF(t, d) = TF(t, d) × IDF(t)
Les mots communs a tous les documents (articles, prepositions) ont un IDF bas, tandis que les mots specialisés ont un IDF eleve.

Intuition : Un mot est important si : (1) il apparat souvent dans ce document ET (2) il est rare dans les autres documents.

</> TF-IDF : implementation Scikit-learn

Scikit-learn fournit `TfidfVectorizer` qui combine tokenization, decompte et TF-IDF en une seule etape.

PYTHON

```python
from sklearn.feature_extraction.text import TfidfVectorizer
import numpy as np

# Documents exemple
docs = [
    "python est un langage de programmation",
    "machine learning utilise python",
    "data science avec machine learning",
    "deep learning et neural networks"
]

# Creer et fit TfidfVectorizer
vectorizer = TfidfVectorizer(
    max_features=20,  # Garder top 20 mots
    stop_words='french',  # Supprimer stopwords
    min_df=1,  # Mot doit appara dans min 1 doc
    max_df=0.8  # Mot ne doit pas appara dans >80% docs
)

X = vectorizer.fit_transform(docs)

# Afficher les features (mots)
features = vectorizer.get_feature_names_out()
print(f"Vocabulaire ({len(features)} mots):", features)

# Afficher les scores TF-IDF pour le premier document
tfidf_scores = X[0].toarray()
for i, score in enumerate(tfidf_scores[0]):
    if score > 0:
        print(f"{features[i]:20} : {score:.4f}")
```

Parametres utiles :

- `max_features` : Limiter a N mots les plus communs (reduit dimensionalite)

- `min_df` : Ignorer les mots tres rares (mots avec typos)
- `max_df` : Ignorer les mots tres frequents (stopwords non-standard)
- `ngram_range` : Considerer les n-grams (bigrammes, trigrammes)

Dense vs Sparse matrices

Les vecteurs de texte sont souvent **sparse** (beaucoup de zeros). Comprendre les deux formats est crucial pour l'efficacite.

Dense Matrix

Tous les elements sont stockes (zeros et non-zeros).

PYTHON

```python
array([
  [0.408, 0.408, 0.408],
  [0.577, 0.000, 0.577],
  [0.000, 0.707, 0.707]
])

# Usage memoire: O(n*m)
# Ideal pour: petit dataset
```

Sparse Matrix

Seuls les elements non-zeros sont stockes.

PYTHON

```python
(0, 0) 0.408
(0, 1) 0.408
(0, 2) 0.408
(1, 0) 0.577
...

# Usage memoire: O(nnz)
```

```
# Ideal pour: grands datasets
```

PYTHON

```python
# Conversion dense <-> sparse
from scipy.sparse import csr_matrix

# Transformer une sparse matrix en dense
X_sparse = vectorizer.fit_transform(docs)  # Retourne sparse
X_dense = X_sparse.toarray()  # Convertir en dense

print(f"Sparse shape: {X_sparse.shape}, nnz: {X_sparse.nnz}")
print(f"Sparsity: {1 - X_sparse.nnz / (X_sparse.shape[0] *
X_sparse.shape[1]):.2%}")

# Les algorithmes ML de Scikit-learn acceptent les deux formats
# Les algorithmes acceptant sparse : LogisticRegression, SVM, etc.
from sklearn.linear_model import LogisticRegression

classifier = LogisticRegression()
classifier.fit(X_sparse, y)  # Fonctionne aussi avec sparse!
```

Best practice : Garder les matrices sparse tant que possible pour economiser la memoire, surtout avec des vocabulaires larges (10000+ mots).

N-grams : capturer le contexte local

Les **N-grams** sont des sequences de N tokens consecutifs. Ils captent le contexte local et les expressions multi-mots.

Exemple: "machine learning est cool"

- **Unigrams (1-grams):**
  - "machine", "learning", "est", "cool"
- **Bigrams (2-grams):**
  - "machine learning", "learning est", "est cool"
- **Trigrams (3-grams):**
  - "machine learning est", "learning est cool"

Avantage des N-grams

- Capture des expressions multi-mots
- Preserv un peu du contexte
- "New York" vs "new", "york"
- "not good" vs "not" + "good" (probleme de negation)

PYTHON

```python
# Utiliser n-grams avec TfidfVectorizer
vectorizer = TfidfVectorizer(
    ngram_range=(1, 2),  # Unigrams + bigrams
    max_features=100
)

X = vectorizer.fit_transform(docs)
```

```
features = vectorizer.get_feature_names_out()

# Afficher les features
for i, f in enumerate(features[:20]):
    print(f"{i+1:2}. {f}")
```

> ⚠ **Cout en performance :** Les N-grams augmentent la dimensionalite. (1,3) avec 1000 mots peut = 30000+ features. A utiliser avec `max_features` .

⚖️ Comparaison : Bag of Words vs TF-IDF

Aspect	Bag of Words	TF-IDF
Definition	Decompte brut des occurrences	Decompte pondere par la rareté
Poids des mots	Egal pour tous	Haut pour mots rares, bas pour communs
Performance	Adequate pour certaines taches	Generalement meilleur
Complexite	Simple	Un peu plus complexe
Taches ideales	Baseline, classification simple	Classification, recherche, clustering
Information perdue	Ordre des mots, contexte	Ordre des mots (sauf avec n-grams)

✅ **Recommandation :** Commencer par TF-IDF. C'est rapide, simple, et donne de bons resultats dans la plupart des cas. N-grams peuvent ameliorer pour certaines taches.

Pipeline complet : preprocessing + vectorization

En pratique, il faut combiner le preprocessing du Module 1 avec la vectorization.

PYTHON

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.pipeline import Pipeline
import re

# Preprocessing custom
def clean_text(text):
    # Minuscules
    text = text.lower()
    # Supprimer ponctuation
    text = re.sub(r'[^a-z\s]', '', text)
    # Espaces extras
    text = re.sub(r'\s+', ' ', text).strip()
    return text

# Appliquer preprocessing avant vectorization
docs = ["Ceci est un exemple!", "Voila un autre exemple."]
docs_cleaned = [clean_text(d) for d in docs]

# Vectorize
vectorizer = TfidfVectorizer(
    max_features=100,
    ngram_range=(1, 2),
    min_df=1,
    max_df=0.9
)

X = vectorizer.fit_transform(docs_cleaned)
print(f"Shape: {X.shape}")  # (2, num_features)
```

> **Important :** Appliquer le preprocessing a tous les documents (train et test) de maniere coherente. Utiliser sklearn `Pipeline` pour automatiser ceci.

Feature engineering avancee

Au-dela des features de base, on peut extraire d'autres signaux utiles du texte.

Features structurelles

- Longueur du texte (nb mots)
- Longueur moyenne des mots
- Nombre de caracteres
- Nombre de phrases
- Ponctuation (?, !, ...)

Features semantiques

- Presence de mots-clés specifiques
- Presence de mots sentiment (bon, mauvais)
- Presence de majuscules (ALL CAPS)
- Presence de mots rares ou speciaux

PYTHON

```python
def extract_features(text):
    # Features structurelles
    nb_words = len(text.split())
    nb_chars = len(text)
    nb_sentences = len(re.split(r'[.!?]', text))
    avg_word_len = nb_chars / nb_words if nb_words > 0 else 0

    # Features semantiques
    has_exclamation = 1 if '!' in text else 0
    has_question = 1 if '?' in text else 0
```

```
    nb_caps = sum(1 for c in text if c.isupper())

    return {
        'nb_words': nb_words,
        'nb_chars': nb_chars,
        'avg_word_len': avg_word_len,
        'has_exclamation': has_exclamation,
        'has_question': has_question,
        'nb_caps': nb_caps
    }

# Combiner avec TF-IDF
from sklearn.preprocessing import StandardScaler

features = [extract_features(d) for d in docs]
# Combiner avec X_tfidf pour un modele plus puissant
```

Exercice : classification de sentiments avec TF-IDF

Challenge

Vous avez un dataset de critiques de films (positives et negatives). Creez un vectoriseur TF-IDF et entrainez un classifieur Logistic Regression.

Etapes :

1. Charger et nettoyer les donnees
2. Vectoriser avec TfidfVectorizer (parametres optimaux)
3. Diviser train/test
4. Entraizer LogisticRegression
5. Evaluer (accuracy, precision, recall)

PYTHON

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
import re

# 1. Exemple de donnees
reviews = [
    "Ce film est genial et passionnant!",
    "Horrible, une perte de temps",
    "Excellent! A regarder absolument",
    "Nul et ennuyeux au maximum"
]
labels = [1, 0, 1, 0]  # 1=positif, 0=negatif

# 2. Vectorisation TF-IDF
vectorizer = TfidfVectorizer(
    max_features=1000,
    ngram_range=(1, 2),
    stop_words='french'
)
X = vectorizer.fit_transform(reviews)
```

```python
# 3. Split train/test
X_train, X_test, y_train, y_test = train_test_split(
    X, labels, test_size=0.2, random_state=42
)

# 4. Entrainement
classifier = LogisticRegression()
classifier.fit(X_train, y_train)

# 5. Evaluation
y_pred = classifier.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2%}")
```

Conclusion et perspectives

Ce que nous avons couvert

- Tokenization et ses differentes granularites
- Bag of Words : approche basique
- TF-IDF : weighting intelligent
- N-grams et contexte local
- Dense vs Sparse matrices
- Feature engineering structurel

→ Prochaines etapes

- Embeddings denses (Word2Vec)
- Modeles neuraux (CNN, RNN)
- Transformers et BERT
- Fusion de multiples sources de features
- Optimization hyperparametres

Key takeaway : TF-IDF + LogisticRegression est une baseline solide et interpretable pour la plupart des taches de classification de texte. C'est rapide et efficace avant d'explorer des architectures plus complexes.